

Real-Time FPGA Acceleration of Magnetic Pendulum Basin-of-Attraction Dynamics

Team Quick Mafs

Department of Electrical and Electronic Engineering
Imperial College London

Project: Year 2 Electronics Design Project
Platform: PYNQ-Z1 FPGA
Academic Year: 2025–2026

Contents

1	Abstract	1
2	Design Development	1
2.1	Motivation	1
2.2	System Overview	2
2.3	Project Management	3
2.4	Sprints	4
3	Mathematical Model and Reference	5
3.1	Continuous-Time Model	5
3.2	Discrete-Time Approximation	6
3.3	Settle and Timeout Conditions	6
3.4	Floating-Point Reference & Fixed-Point Model	6
3.5	Parameter Selection	7
3.6	Final-State Sensitivity	9
3.7	Parameter Sweep and Fractional Dimension	9
3.8	Potential-Well and 3D Trajectory Visualisation	10
4	Accelerator Microarchitecture	12
4.1	Pixel Scanner & Coordinate Mapper	12
4.2	Fixed-Point Arithmetic Strategy	12
4.3	Pipelined Pendulum Solver	12
4.3.1	Main Solver Lane	13
4.3.2	Pipeline Stage Organisation	13
4.3.3	Pipelined Fixed-Point Multiplier	13
4.3.4	Constant Timestep Scaling	14
4.4	Settling Condition Detection	15
4.5	Band Streaming for High-Resolution Output	15
4.6	Overall Implementation of One Lane	15
4.7	Parallel Solver Architecture	17
4.8	AXI-Lite Control Interface	18
4.9	AXI-DMA Data Transfer	18
4.10	Trajectory Capture and Plotting	18
5	Interactive Visualisation & Control	20
5.1	Physical Interaction Layer: ArUco Magnet Tokens	20
5.1.1	Initial Tracking Approach: IMU-Based Tokens	20
5.1.2	Final Tracking Approach: Camera-Based ArUco Detection	21
5.1.3	ArUco Marker CV Pipeline	22
5.2	Python/PYNQ Control Software	24
5.2.1	Hardware Interface	24
5.2.2	DMA Transfer	24
5.3	Unity Based User Interface	25
5.3.1	Networking Layer and Wire Protocol	25
5.3.2	Parameter Control Interface	26
5.3.3	Basin Visualisation	27
5.3.4	Physical Token Tracking Integration	27
5.3.5	Navigation & View Management	27
5.3.6	Design Decisions & Trade-offs	28
6	Verification and Evaluation	29
6.1	Numerical Accuracy Evaluation	29
6.1.1	Match Rate Analysis	29
6.2	Hardware Feasibility Evaluation	31
6.2.1	Timing Closure	31

6.2.2	Resource Utilisation	31
6.3	Runtime Performance Evaluation	32
6.3.1	Cross-Platform Performance Comparison	32
6.4	Resolution and Latency Trade-Off	34
6.4.1	Resolution Scaling	34
6.4.2	End-to-End Latency	34

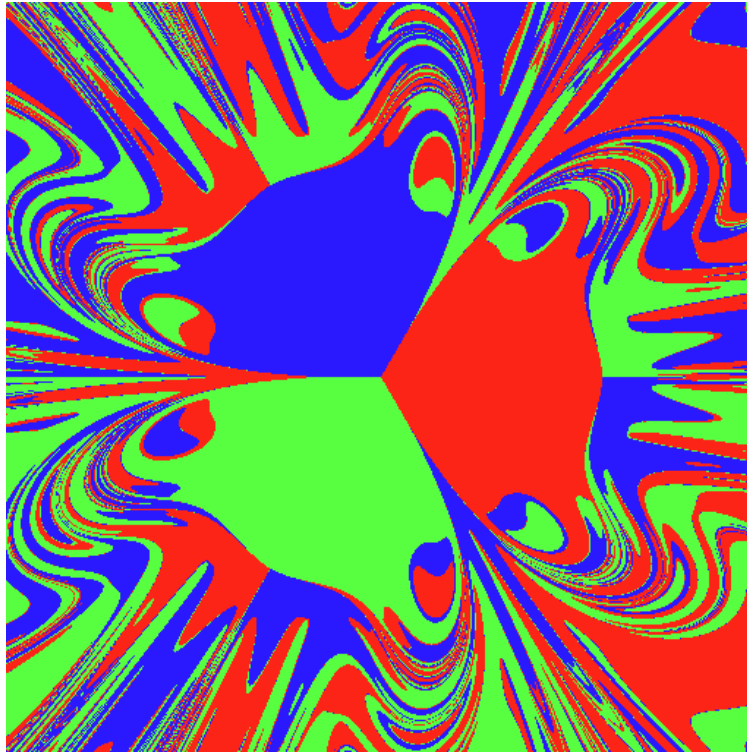


Figure 1: Fractal image

1 Abstract

This report presents the design and implementation of a magnetic pendulum visualisation system. The system computes the Basin-of-Attraction (BoA) of a damped magnetic pendulum, where each pixel corresponds to an initial position and is classified according to the magnet it settles near. During simulation, each trajectory is independent, this makes the problem embarrassingly parallel and well suited to FPGA implementation.

The magnetic pendulum solver is implemented on the PYNQ-Z1 board using a modular and parameterised architecture. The design maps the basin-of-attraction computation into scalable pipelines, with parallelism constrained primarily by the available FPGA resources. A pixel scanner and coordinate mapper generate the initial conditions for each pixel, while a multi-stage pipelined parallel solver performs semi-implicit Euler updates to increase throughput.

The software model acts as a reference for the system. It shows how different starting conditions evolve, and guide design decisions. The implementation results are then compared with this model to check that they match the expected behaviour and to measure accuracy.

The FPGA computation is connected to a Unity visualisation interface to present the results in an interactive and accessible form. Users can observe the generated basin-of-attraction, adjust simulation parameters, and trace the pendulum trajectories.

2 Design Development

2.1 Motivation

Alternative candidate algorithms were evaluated during the initial project selection stage, including Mandelbrot fractal generation, neural-network-based visual processing, and three-dimensional fluid simulation. Mandelbrot generation is visually appealing and highly parallelisable but offers limited modelling depth. Neural-network approaches introduce additional dependencies such as model selection and training, shifting the emphasis away from FPGA architecture and real-time visualisation. Three-dimensional fluid simulation is computationally complex, but its memory-bandwidth demands exceed the PYNQ-Z1's budget.

The magnetic pendulum was selected because it offers the best balance between mathematical depth, hardware feasibility, and alignment with the requirements. It preserves pixel-level parallelism since each pixel corresponds to an independent initial condition. It also introduces richer system-level challenges, including physical modelling and numerical integration. This makes it appropriate for demonstrating FPGA acceleration and a meaningful educational visualisation.

2.2 System Overview

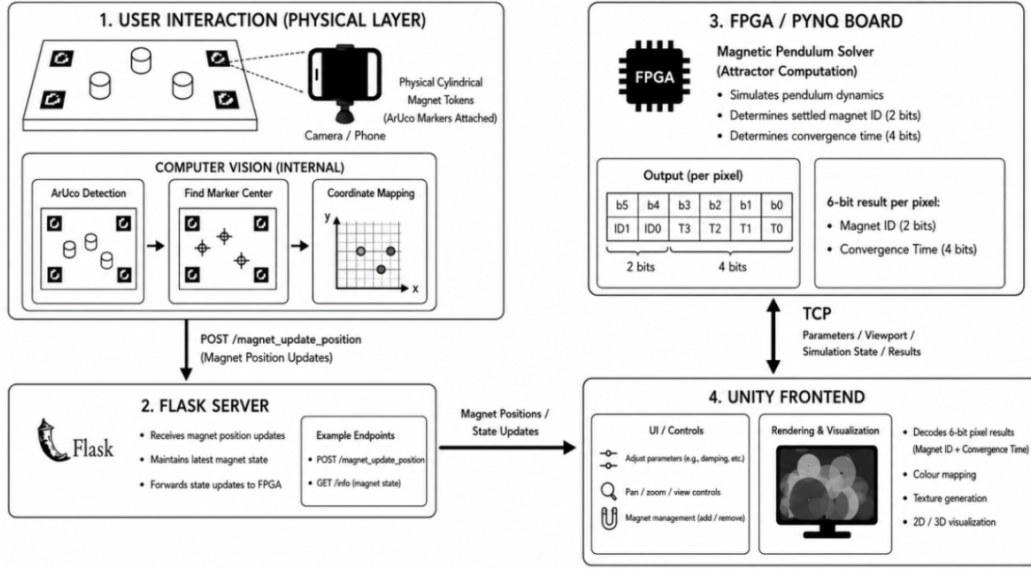


Figure 2: System overview of the magnetic pendulum accelerator.

As shown in Figure 2, the system is organised as four interconnected layers: a physical interaction layer, a computer-vision and server layer, an FPGA acceleration layer, and a Unity visualisation layer. The physical layer uses cylindrical magnet tokens with attached ArUco markers. A camera observes these tokens, and the computer-vision pipeline detects the markers, estimates their centres, and maps their image-space positions into the simulation coordinate system. These magnet positions are then sent to a Flask server, which maintains the latest shared simulation state.

The PYNQ-Z1 board receives the simulation parameters and magnet positions through the software control layer. The FPGA accelerator computes the basin-of-attraction by simulating the magnetic pendulum dynamics for each pixel in the selected coordinate window. Each output pixel is encoded as a 6-bit result containing the final magnet label and a convergence-time category. The completed frame is transferred from the programmable logic to the PYNQ processing system using AXI-DMA, then forwarded to the Unity frontend over TCP.

Unity decodes the received pixel data, applies colour mapping and texture generation, and renders the basin-of-attraction in both 2D and 3D views. The user interface also allows the user to adjust simulation parameters, inspect the basin through panning and zooming, and update the magnet configuration through the physical token system. This creates a closed-loop workflow in which physical magnet placement, computer vision, FPGA acceleration, and interactive visualisation operate together as a complete end-to-end system.

This architecture deliberately separates low-bandwidth control data from high-throughput frame data: parameter and magnet updates are handled through lightweight control paths, while dense image results are transferred using DMA and TCP. This separation reduces software overhead and allows the FPGA accelerator to focus on the computationally intensive basin-generation task.

2.3 Project Management

The team adopted an iterative project-management approach centred on reaching an early minimum viable product (MVP), followed by progressive optimisation and feature expansion. This was important because the project involved several tightly connected subsystems. Establishing a working end-to-end path early reduced integration risk and allowed later development to focus on improving performance, accuracy, and user interaction.

To manage the project effectively, the group was divided into three sub-teams aligned with the system architecture: Hardware, Software, and User Interface. The hardware team worked on the FPGA design, the software team developed the PYNQ control and communication systems, and the UI team created the Unity frontend and user interactions. Weekly in-person meetings were used to review progress, identify blockers, and coordinate integration tasks, while online calls supported rapid debugging when cross-subsystem issues occurred.

Separate Discord channels were created for each sub-team so that detailed technical discussions could take place efficiently, while remaining visible to the whole group. The GitHub repository was used as the central collaboration platform, with commits and code changes tracked throughout development. This structure helped the team maintain clear ownership of individual subsystems while still coordinating shared interfaces.

Table 1: Subsystem ownership and main contributions.

Name	Main Contributions
Krish	Physical interaction layer; ArUco marker setup; magnet-token interface; Unity input integration.
Josh	Flask shared-state server; camera streaming; Unity rendering support; backend/frontend integration.
David	Problem formulation; software experiments and numerical analysis; FPGA submodule implementation and verification.
Chen	Nearest-magnet and settling-detection logic; top-level FPGA integration; 2D trajectory capture and visualisation support.
Yichan	FPGA solver datapath; BRAM/LUT setup; synthesis and timing closure; hardware–UI TCP integration.
Philippa	CPU baseline implementation and comparison; coordinate mapper; 3D potential trajectory visualisation.

Through collaborative debugging and optimisation during integration, the team evaluated the final system against the predefined requirements and resolved major cross-subsystem issues.

2.4 Sprints

An Agile development approach based on focused sprints was used for effective project management. Each sprint included clear and achievable objectives that helped move the project toward a fully integrated and tested system. Sprint plans were adapted when necessary.

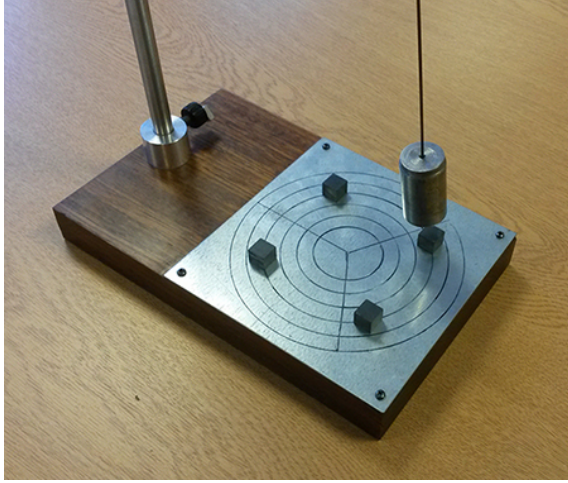
Table 2: Sprint plan and main development outcomes.

Sprint	Period	Focus	Main Outcomes
Sprint 1	Week of 18/05	Modelling and architecture	Defined the magnetic pendulum model, selected initial parameter ranges, built the Python floating-point reference, and proposed the initial FPGA–Flask–Unity system architecture.
Sprint 2	Week of 25/05	End-to-end MVP	Implemented the first single-lane FPGA solver and established a basic working path between the FPGA, PYNQ software, Flask server, and Unity frontend.
Sprint 3	Week of 31/05	Hardware scaling and numerical design	Extended the solver to a multi-lane hardware architecture, completed the fixed-point software model, and selected the Q-format and LUT structure for the FPGA datapath.
Sprint 4	Week of 08/06	Integration and optimisation	Replaced the initial Flask-based image-transfer path with direct Unity–PYNQ TCP communication, added trajectory visualisation and final-state sensitivity features, and performed match-rate analysis.
Sprint 5	Week of 15/06	Validation and final polish	Completed final hardware optimisations, runtime performance, improved the user interface, and prepared the final integrated system for demonstration.

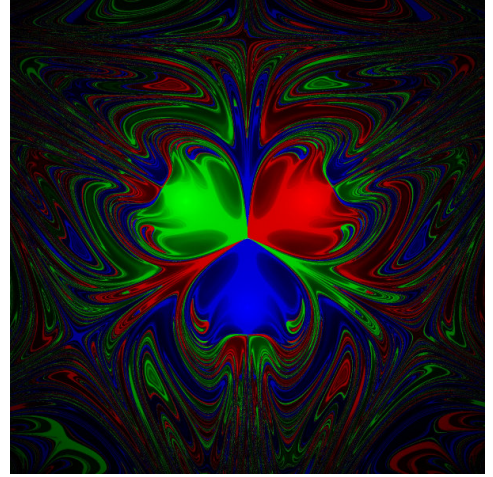
3 Mathematical Model and Reference

3.1 Continuous-Time Model

The magnetic pendulum consists of a pendulum bob suspended above a plane containing several magnets. When released from an initial position, the bob moves under the combined effects of forces. Over time, energy is dissipated and the bob eventually settles near one of the magnets. By assigning a different colour to each final magnet, the system visualises how different initial positions lead to different long-term outcomes.



(a) Magnetic pendulum setup



(b) Basins of Attraction

Figure 3: Magnetic pendulum and its corresponding basin of attraction.

To capture the system's essential behaviour, the model includes three dominant effects: the pendulum's natural restoring motion, dissipative damping such as air resistance, and the attractive forces by the magnets in the base plane. In the full physical system, the bob moves in three dimensions along the lower surface of a sphere centred at the suspension point with radius equal to the string length. However, directly integrating spherical-pendulum equations would add unnecessary computational complexity. The motion is thus modelled in the horizontal plane, where the projected bob position at time t is represented by $\mathbf{r}(t) = (x(t), y(t))$ and the desired output is a top-down BoA.

The equation of motion can be modelled as:

$$\ddot{\mathbf{r}} = -\gamma \dot{\mathbf{r}} - \omega_0^2 \mathbf{r} + \sum_{i=1}^N \mu_i \frac{\mathbf{m}_i - \mathbf{r}}{(\|\mathbf{m}_i - \mathbf{r}\|^2 + h^2)^{3/2}}.$$

where $\mathbf{r}$ represents the horizontal position of the bob while $\dot{\mathbf{r}}$ and $\ddot{\mathbf{r}}$ denote its velocity and acceleration. The remaining parameters include the damping factor γ , the restoring strength ω_0^2 , the strength of the i -th magnet μ_i , the magnet position $\mathbf{m}_i = (X_i, Y_i)$, and the vertical separation h between the bob and the magnet plane. The denominator appears because the magnetic force becomes stronger when the bob is closer to the magnet, in which h^2 prevents the force from becoming infinite when the bob is directly above a magnet.

The continuous-time model is expressed in state-space form using the horizontal position and velocity of the bob, where the state is $\mathbf{s}(t) = (x(t), y(t), v_x(t), v_y(t))^T$. Each grid point in the output image corresponds to a different initial position (x_0, y_0) , and the bob is released from rest so that $v_x(0) = v_y(0) = 0$. The future trajectory is then obtained by integrating the nonlinear equations of

motion, the velocity equations are:

$$\begin{aligned}\dot{v}_x &= -\gamma v_x - \omega_0^2 x + \sum_{i=1}^N \mu_i \frac{X_i - x}{((X_i - x)^2 + (Y_i - y)^2 + h^2)^{3/2}}, \\ \dot{v}_y &= -\gamma v_y - \omega_0^2 y + \sum_{i=1}^N \mu_i \frac{Y_i - y}{((X_i - x)^2 + (Y_i - y)^2 + h^2)^{3/2}}.\end{aligned}$$

This formulation converts the magnetic pendulum simulation into an initial-value problem: for each pixel, the system evolves from its corresponding initial condition until the bob either settles near a magnet or reaches the maximum iteration limit.

3.2 Discrete-Time Approximation

The continuous-time equation of motion is nonlinear and does not provide a closed-form expression for the final magnet. Therefore, each trajectory must be advanced numerically using a discrete-time integration scheme. Let $\mathbf{r}_n = (x_n, y_n)$ and $\mathbf{v}_n = (v_{x,n}, v_{y,n})$ denote the position and velocity of the bob at timestep t_n , and let $\mathbf{a}_n = (a_{x,n}, a_{y,n})$ be the acceleration evaluated from the current state.

A semi-implicit Euler method was selected for the simulation and hardware implementation. In this scheme, the velocity is updated first using the current acceleration $\mathbf{v}_{n+1} = \mathbf{v}_n + \mathbf{a}_n \Delta t$, this new velocity is then used to update the position, $\mathbf{r}_{n+1} = \mathbf{r}_n + \mathbf{v}_{n+1} \Delta t$. This update sequence converts the continuous trajectory into a finite sequence of states that can be evaluated iteratively for each initial condition.

The semi-implicit Euler method was chosen over high-order methods like RK4 because it requires only one acceleration evaluation per timestep, mapping cleanly onto the fixed-point datapath without the additional DSP overhead that multiple evaluations would demand, while still avoiding the artificial energy growth associated with explicit Euler integration.

3.3 Settle and Timeout Conditions

The simulation must decide when the pendulum has settled near a magnet. At each time step, we find the nearest magnet:

$$j = \arg \min_i \|\mathbf{r}_n - \mathbf{m}_i\|.$$

The pendulum can be considered settled near magnet j if two conditions are satisfied:

$$\|\mathbf{r}_n - \mathbf{m}_j\| < r_{\text{settle}},$$

and

$$\|\mathbf{v}_n\| < v_{\text{settle}}.$$

Here, r_{settle} is a small distance threshold and v_{settle} is a small velocity threshold. The first condition checks that the bob is close to a magnet, the second condition checks that the bob is moving slowly enough to be considered settled. To avoid stopping too early due to a temporary slow-down, the conditions are required to hold for several consecutive time steps. If the pendulum does not settle after a maximum number of steps, the initial condition is marked as unresolved.

3.4 Floating-Point Reference & Fixed-Point Model

The floating-point model is used as the mathematical “golden reference” for the magnetic pendulum system. It defines the nominal BoA image before hardware constraints are introduced.

The reference model serves two primary functions. First, it establishes a baseline for selecting appropriate simulation parameters. Second, it provides the benchmark against which the fixed-point

and FPGA implementations are evaluated. Since the floating-point model is free from quantisation effects and other hardware-induced inaccuracies, any discrepancies between the FPGA output and the reference image can be attributed to the numerical effects introduced by the hardware implementation.

The fixed-point model was developed as an RTL-aligned software reference rather than an independent mathematical model. Its purpose is to reproduce the arithmetic behaviour of the final hardware implementation, including fixed-point scaling, LUT (LookUp-Table) force approximation, and truncation / rounding logic. This provides a bridge between the ideal floating-point reference and the SystemVerilog design: the floating-point model defines the expected physical behaviour, while the fixed-point model checks whether the hardware-oriented arithmetic still produces an acceptable BoA result. Detailed design choices such as Q-format selection, LUT structure, bit growth, and pipeline organisation are discussed in section 4 and 6.

3.5 Parameter Selection

The parameter selection process was guided by the objective of generating a visually meaningful BoA map within a practical FPGA computation budget. The aim of the model is not to reproduce the physical pendulum trajectory indefinitely, but to define a reliable and reproducible reference configuration for hardware acceleration. The selected parameters therefore balance three requirements: preserving the characteristic basin structure, maintaining numerical stability, and limiting the computational cost per pixel.

The parameters can be divided into four groups. The first group consists of physical model parameters $\theta = \{\gamma, \mu, h, \omega_0^2\}$ and magnet positions. These parameters determine the qualitative behaviour of the pendulum. The second group consists of numerical simulation parameters, including the timestep Δt , maximum iteration count, settle thresholds r_{settle} and v_{settle} . These do not change the continuous-time model itself, but determine how the finite simulation is terminated. The third group consists of visual and system parameters, such as image resolution and coordinate window. The final group consists of hardware approximation parameters, including fixed-point format and LUT size.

To produce a clear three-region BoA visualisation, we place the three magnets in an equilateral triangle around the origin. A nominal physical parameter set $\theta_{\text{nom}} = \{0.2, 1, 0.5, 1\}$ was selected to provide a stable baseline for later experiments.

The numerical termination parameters were selected using floating-point experiments. Very strict thresholds such as $r_{\text{settle}} = 0.10$ and $v_{\text{settle}} = 0.01$ produced too many unresolved pixels for a useful display. Increasing the thresholds improved the settled fraction, but overly loose velocity thresholds caused visible distortion by terminating trajectories prematurely. The selected values $r_{\text{settle}} = 0.25$, $v_{\text{settle}} = 0.05$ provided a practical compromise: most pixels settled while the visible basin structure was preserved.

The maximum iteration count was selected separately from the settle thresholds. Increasing the maximum step count from 4000 to much larger values did not fully remove the central unresolved region, indicating that these pixels were not simply slow to converge. Instead, they failed the chosen magnet-based settling condition, often because the symmetry of the three-magnet configuration keeps trajectories near the centre rather than close to a particular magnet. These unresolved pixels were therefore assigned to the nearest magnet when maximum iteration count is reached. A maximum step count of 4000 was selected because it is sufficient for the pixels that strictly settle under the chosen criterion, while also fitting exactly within a 12-bit hardware counter.

The timestep was chosen by comparing the nominal value $\Delta t = 0.01$ with a smaller value of $\Delta t = 0.001$. To make the comparison fair, the maximum number of steps and the consecutive settling requirement were scaled so that the total simulated time and settling hold time remained approximately constant. The resulting basin maps were visually equivalent, showing that $\Delta t = 0.01$

is sufficiently accurate for the intended visualisation. This value was therefore selected because it reduces the number of required iterations by approximately a factor of ten compared with $\Delta t = 0.001$, which directly improves FPGA throughput.

The computational resolution was selected as 180×180 , which is small enough for hardware acceleration while still preserving the main basin structure. The coordinate window was chosen as $[-1.8, 1.8] \times [-1.8, 1.8]$, which contains the magnet arrangement, keeps the origin at the centre of the image, and leaves enough surrounding space to show the basin boundaries.

The final nominal parameter set used for the floating-point reference and subsequent hardware comparison is summarised in Table 3.

Table 3: Selected nominal parameters for the magnetic pendulum reference model.

Parameter	Selected value	Purpose
Resolution	180×180	Hardware-oriented computation target
Coordinate window	$[-1.8, 1.8] \times [-1.8, 1.8]$	Symmetric visualisation domain
γ	0.20	Damping coefficient
ω_0^2	1.0	Central restoring strength
μ	1.0	Magnet strength scale
h^2	0.25	Avoids singular magnetic force
Δt	0.01	Numerical timestep
r_{settle}	0.25	Distance threshold for settlement
v_{settle}	0.05	Velocity threshold for settlement
Maximum steps	4000	Runtime limit and 12-bit hardware counter

Floating-point nominal reference, 180×180 , max steps = 4000

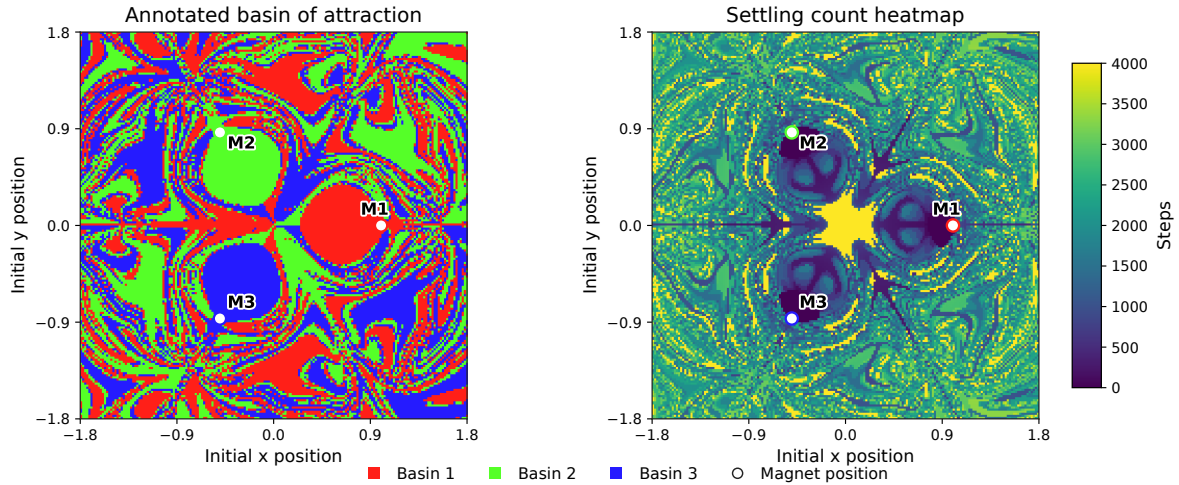


Figure 4: Floating-point nominal reference results for the magnetic pendulum with a 180×180 grid and maximum steps set to 4000. The left panel shows the settling count heatmap, while the right panel shows the annotated basin of attraction.

3.6 Final-State Sensitivity

The magnetic pendulum is a deterministic nonlinear dynamical system whose long-term behaviour can be highly sensitive to small changes in initial conditions. Although the system contains no randomness, neighbouring initial conditions may converge to different final magnets, especially near the complicated boundaries between basins. This property is known as final-state sensitivity (FSS), and it provides a quantitative measure of the system's unpredictability.

A basic FSS test compares the final magnet reached from an initial condition with the final magnets reached from slightly perturbed initial conditions. For a perturbation scale $\epsilon \ll \mathcal{O}(1)$, the system simulates

$$\mathbf{x}_0 = (x_0, y_0), \quad \mathbf{x}_{0+\epsilon} = (x_0 + \epsilon, y_0), \quad \mathbf{x}_{0-\epsilon} = (x_0 - \epsilon, y_0).$$

If all three trajectories settle at the same magnet, the pixel is classified as stable. If the final magnet changes under the perturbation, the pixel is classified as final-state sensitive. The fraction of sensitive pixels,

$$f_\epsilon = \frac{N_\epsilon}{N_\Gamma},$$

where N_Γ is the total number of sampled initial conditions and N_ϵ is the number classified as sensitive, gives a measure of how unpredictable the basin structure is at scale ϵ .

Consider a set of initial conditions, Γ , containing a total of N_Γ points, of which N_ϵ are susceptible to FSS. In many systems, these quantities obey a power-law relation,

$$\frac{N_\epsilon}{N_\Gamma} = f_\epsilon \sim \epsilon^\alpha,$$

where $0 < \alpha < 1$ is known as the uncertainty exponent. For a two-dimensional domain Γ , the uncertainty exponent is commonly written as $\alpha = 2 - D$, where D denotes the uncertainty dimension. A simple estimate then gives

$$D = 2 - \frac{d \log_{10}(f_\epsilon)}{d[\log_{10}(\epsilon)]},$$

with $1 < D < 2$. Smaller values of α , which correspond to values of D approaching 2, indicate stronger sensitivity and increasingly fractal basin boundaries. Thus, the extent of FSS in a system is directly connected to the complexity of the boundary patterns within its basins. By contrast, regular, non-fractal basin boundaries are characterized by $\alpha = 1$.

This concept extends the basic basin generator into a multi-scale FSS accelerator, suitable for chaotic dynamics analysis. Instead of testing only one perturbation size, the system evaluates several ϵ values to investigate how sensitivity changes across spatial scales, producing basin maps, sensitivity maps, iteration-count heatmaps, and sensitivity-versus-scale data. Users can observe the interaction between nonlinear mathematical modelling and quantitative measurement of unpredictability.

3.7 Parameter Sweep and Fractional Dimension

As an example of a one-parameter-at-a-time sweep, we vary the damping factor γ while keeping all other physical and numerical parameters fixed at their nominal values. For each damping value

$$\gamma_1, \gamma_2, \dots, \gamma_L,$$

the accelerator generates the corresponding basin map and final-state sensitivity data. The sensitive-pixel fraction $f_\epsilon(\gamma)$ is computed for each value of γ , and the relationship between f_ϵ and the perturbation scale ϵ can be plotted on a log-log scale. From the slope of this curve, the uncertainty dimension can be estimated as

$$D(\gamma) = 2 - \frac{d \log_{10}(f_\epsilon(\gamma))}{d[\log_{10}(\epsilon)]}.$$

This sweep allows us to quantify how damping affects the complexity of the basin boundaries. Physically, a larger damping factor removes energy from the pendulum more quickly, causing trajectories to settle sooner and reducing the opportunity for long chaotic wandering between magnets. Therefore, we expect stronger damping to reduce the density of fine basin-boundary structure. This should appear mathematically as a lower sensitive-pixel fraction $f_\epsilon(\gamma)$, a lower average settling time, and a smaller uncertainty dimension $D(\gamma)$. In this way, the damping sweep provides a controlled method for connecting a physical parameter of the model to both the visual complexity of the basin map and the quantitative final-state sensitivity of the system.

3.8 Potential-Well and 3D Trajectory Visualisation

Although the FPGA updates the pendulum state directly from force and velocity equations, the same force model admits an equivalent potential-well interpretation. This provides a useful visual explanation of the BoA result: magnets correspond to local wells in the landscape, and damping causes each trajectory to lose energy until it settles into one of them. The potential surface is therefore introduced only as an interpretive tool for visualising individual trajectories, not as an additional quantity required by the FPGA solver.

The magnetic force in the model can be interpreted using the potential-energy function

$$V(x, y) = \frac{1}{2}\omega_0^2(x^2 + y^2) - \mu \sum_i \frac{1}{\sqrt{(X_i - x)^2 + (Y_i - y)^2 + h^2}}.$$

The first term is the central restoring potential penalising large displacement from the origin. The second term is the magnetic contribution. Since this term becomes more negative near a magnet, each magnet creates a local potential well.

Taking the negative gradient of $V(x, y)$ gives the restoring and magnetic force components:

$$\begin{aligned} -\frac{\partial V}{\partial x} &= -\omega_0^2 x + \mu \sum_i \frac{X_i - x}{((X_i - x)^2 + (Y_i - y)^2 + h^2)^{3/2}}, \\ -\frac{\partial V}{\partial y} &= -\omega_0^2 y + \mu \sum_i \frac{Y_i - y}{((X_i - x)^2 + (Y_i - y)^2 + h^2)^{3/2}}. \end{aligned}$$

Therefore, ignoring damping, the acceleration due to the restoring and magnetic forces can be written as

$$\begin{bmatrix} a_x \\ a_y \end{bmatrix} = -\nabla V(x, y).$$

Including damping, the full acceleration model used in the simulation is

$$\begin{aligned} a_x &= -\gamma v_x - \omega_0^2 x + \mu \sum_i \frac{X_i - x}{((X_i - x)^2 + (Y_i - y)^2 + h^2)^{3/2}}, \\ a_y &= -\gamma v_y - \omega_0^2 y + \mu \sum_i \frac{Y_i - y}{((X_i - x)^2 + (Y_i - y)^2 + h^2)^{3/2}}. \end{aligned}$$

The damping term $-\gamma \mathbf{v}$ is not part of the potential energy because it depends on velocity rather than position. Instead, it removes mechanical energy from the system over time, allowing the trajectory to settle into one of the potential wells.

The total mechanical energy can be written as

$$E(t) = \frac{1}{2} (v_x(t)^2 + v_y(t)^2) + V(x(t), y(t)).$$

For the continuous-time model with linear damping,

$$\frac{dE}{dt} = -\gamma (v_x(t)^2 + v_y(t)^2) \leq 0.$$

Thus, damping causes the trajectory to lose energy in the ideal continuous-time model until the bob is eventually trapped in the potential well of the final magnet reached by the trajectory.

This motivates the 3D trajectory visualisation. The horizontal plane represents the same x - y coordinates used for the BoA map, while the vertical axis represents the potential value $V(x, y)$. For a selected initial condition (x_0, y_0) , the simulated trajectory gives a sequence of positions

$$(x_0, y_0), (x_1, y_1), \dots, (x_N, y_N).$$

The corresponding 3D trajectory is plotted as

$$(x_n, y_n, V(x_n, y_n)), \quad n = 0, 1, \dots, N.$$

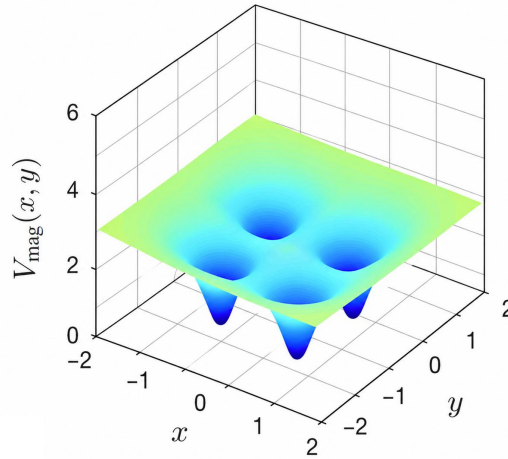


Figure 5: Magnetic potential landscape used to visualise the attraction wells created by the magnets.

This does not represent the physical three-dimensional motion of the pendulum bob. Instead, it plots the planar trajectory on the scalar surface $V(x, y)$, showing how a selected initial condition evolves toward its final attractor.

4 Accelerator Microarchitecture

4.1 Pixel Scanner & Coordinate Mapper

The coordinate mapper converts each discrete image pixel into the corresponding continuous-valued initial position for the magnetic pendulum model. Let r and c denote the horizontal and vertical pixel indices, where $0 \leq r < W$ (image width pixels) and $0 \leq c < H$ (image height pixels). The computational domain is defined by the coordinate limits $[x_{\min}, x_{\max}]$ and $[y_{\min}, y_{\max}]$, with x_{step} and y_{step} specifying the spacing between neighbouring sampled initial conditions. Each pixel (r, c) is therefore mapped to an initial pendulum position (x, y) as

$$\begin{aligned} x_0 &= x_{\min} + r \cdot x_{\text{step}}, & x_{\text{step}} &= \frac{x_{\max} - x_{\min}}{W}, \\ y_0 &= y_{\min} + c \cdot y_{\text{step}}, & y_{\text{step}} &= \frac{y_{\max} - y_{\min}}{H}. \end{aligned}$$

This mapping allows the FPGA to scan the image grid systematically, with each pixel representing one independent initial condition. Using the same coordinate mapping in the floating-point model, fixed-point model, and RTL implementation ensures that all versions of the system evaluate the same set of starting positions, making their BoA outputs directly comparable. The generated fixed-point coordinates and pixel ID are then registered and passed to the next stage of the hardware pipeline.

Rather than instantiating dedicated multipliers for the $r \cdot x_{\text{step}}$ and $c \cdot y_{\text{step}}$ products, the coordinate mapper exploits the fact that the pixel indices p and q are small integers. Each product is computed using a shift-and-add expansion over the bits of the index, for example

$$r \cdot x_{\text{step}} = \sum_i r[i] (x_{\text{step}} \ll i).$$

This implements the multiplication entirely in LUT fabric and preserves the available DSP resources for the computation-heavy solver datapath.

4.2 Fixed-Point Arithmetic Strategy

Fixed-point arithmetic was used to reduce resource usage and support a multi-lane FPGA design. Since the pendulum state remains within a bounded range, floating-point precision is unnecessary for the selected simulation window. The 18-bit fixed-point format also maps efficiently onto the DSP48E1 multipliers on the Zynq-7000 device and provides deterministic arithmetic for comparison with the fixed-point software model.

In the final design, signed Q4.14 is used for most physical quantities in the solver datapath, while unsigned Q5.13 format is used for the squared-distance term q . This separation keeps the main state representation compact while providing additional range for non-negative distance quantities. The detailed numerical evaluation of this choice is presented in Section ??.

4.3 Pipelined Pendulum Solver

Within each timestep update, the pendulum solver is implemented as a feed-forward fixed-point pipeline. Each pass through the lane advances one pixel trajectory by one timestep. The lane computes the acceleration from the magnetic attraction, damping, and restoring terms, then applies the semi-implicit Euler update to produce the next velocity and position. This structure maps efficiently to FPGA hardware, as the update can be decomposed into additions, multiplications, LUT reads, and registered pipeline stages.

4.3.1 Main Solver Lane

The main solver lane, implemented in `lane_main.sv`, performs one timestep update for a single pixel trajectory. It receives the current pendulum state and pixel metadata, and outputs the updated state together with the information required for recirculation or frame-buffer writeback.

The update begins by computing the displacement from the bob to each magnet, where $\Delta x_i = x_i - x$ and $\Delta y_i = y_i - y$. The squared-distance term for each magnet is then evaluated as $q_i = \Delta x_i^2 + \Delta y_i^2 + h^2$. These q_i values are used both for nearest-magnet selection and for addressing the inverse-distance LUT. The LUT output approximates $q_i^{-3/2}$, which is multiplied by the displacement terms to obtain the magnetic force contribution. The magnetic force is then combined with the damping and restoring terms to form the acceleration, after which the velocity and position are updated using the semi-implicit Euler equations. Splitting this computation across registered stages improves timing while allowing the pipeline to accept a new pixel state every clock cycle once full.

4.3.2 Pipeline Stage Organisation

The lane is organised as a registered pipeline, with data flowing strictly forward through the arithmetic stages. The pipeline is divided so that no stage contains more than one DSP multiplication or a small amount of combinational logic. This limits the critical path and allows the design to meet the target clock frequency.

A `valid` signal is propagated alongside the arithmetic data so that invalid start-up values are flushed automatically. Pixel metadata, including the pixel ID, step count, settle count, and nearest magnet ID, is also carried through matching registers. This ensures that each output remains associated with the correct trajectory after passing through the pipeline.

This design trades latency for throughput. Although one timestep update takes multiple clock cycles to pass through the full lane, the pipeline can accept a new pixel state every clock cycle once full. Across the multi-lane implementation, this allows several independent pixel trajectories to be advanced in parallel. Table 4 summarises the stage-by-stage organisation of one solver lane and the estimated DSP usage in each stage.

4.3.3 Pipelined Fixed-Point Multiplier

The fixed-point multiplier, implemented in `fx_mul.sv`, is reused throughout the solver datapath. Since the main datapath uses signed Q4.14 operands, each multiplication produces a full-precision Q8.28 intermediate product. This product must then be rescaled back to Q4.14 before being passed to the next pipeline stage.

The multiplier output is computed as

$$C = \text{sat}_{\text{Q4.14}} \left(\left\lfloor \frac{AB + 2^{F-1}}{2^F} \right\rfloor \right), \quad F = 14,$$

where A and B are the integer representations of the two Q4.14 inputs, AB is the 36-bit raw product, and $\text{sat}_{\text{Q4.14}}(\cdot)$ clips the result to the representable Q4.14 range. The addition of 2^{F-1} implements round-to-nearest before the arithmetic right shift, reducing the systematic bias that would be introduced by simple truncation.

Saturation is used instead of two's-complement wraparound. This is important because the magnetic force term can become large when the bob passes close to a magnet. If overflow were allowed to wrap, the sign of a large force could be reversed, causing an unphysical trajectory and potentially changing the final basin classification. Saturation instead limits the value to the closest representable extreme, which is a safer approximation for this system.

To meet timing constraints, the multiplier is divided into two pipeline stages. The first stage registers the raw product using a DSP48E1 multiplier, while the second stage performs rounding,

Table 4: Pipeline Stages and Resource Estimation

Stage	Purpose	DSP Number
S1a	Compute dx & dy	0
S2a	Queue Multiply	2
S2b	Retrieve dx^2 & dy^2	0
S3a	Compute $q = dx^2 + dy^2 + h^2$	0
S3b	Nearest Magnet Select	0
S3c	Settle Check	0
S4a	LUT Lookup	0
S4b	LUT Buffer	0
S5a	Queue Multiply	6 (2 per magnet)
S5b	Retrieve $dx/q^{1.5}$ & $dy/q^{1.5}$	0
S6a	Queue Multiply of $\gamma \times v$, $\omega^2 \times x$, $\text{sum}(dx/q^{1.5})$	6 (2 per magnet)
S6b	Retrieve $\gamma \times v$, $\omega^2 \times x$	0
S6c	Queue Multiply of $\mu \times \text{sum}(dx/q^{1.5})$	6 (2 per magnet)
S6d	Retrieve $\mu \times \text{sum}(dx/q^{1.5})$	0
S6e	Subtract to find a	0
S7a	Queue scale $dt(0.01) \times a$	0
S7b	Retrieve $dt \times a$	0
S7c	Update Velocity	0
S8a	Queue scale $dt(0.01) \times v_{\text{new}}$	0
S8b	Retrieve $dt \times v_{\text{new}}$	0
S8c	Update Position	0

rescaling, and saturation. This separates the DSP multiplication from the saturation comparators, reducing the critical path and improving timing closure. The additional cycle of latency is handled by propagating the corresponding state variables and pixel metadata through matching pipeline registers. The numerical effect of the rounding policy is evaluated later in the verification section.

4.3.4 Constant Timestep Scaling

Several stages in the semi-implicit Euler update require multiplication by the timestep $\Delta t = 0.01$. Since this coefficient is fixed in the final hardware configuration, selected timestep multiplications are implemented using a specialised shift-and-add constant multiplier, `fx_divide_by_100.sv`, rather than a full DSP multiplier.

The constant is approximated as $0.01 \approx (41944/2^{22})$, where $41944 = 2^{15} + 2^{13} + 2^{10} - 2^5 - 2^3$. Therefore, multiplication by the timestep can be implemented as

$$c = \text{sat}_{\text{Q4.14}} \left(\frac{41944a + 2^{21}}{2^{22}} \right).$$

This replaces a DSP multiplication with shifts and additions in the fabric. The approximation error is much smaller than the Q4.14 quantisation step, while the DSP saving provides additional resource headroom for increasing number of parallel solver lanes from 9 to 12.

4.4 Settling Condition Detection

The settle conditions hold simultaneously for three consecutive timesteps before a pixel is declared settled. The distance condition is implemented as $q_{\min} < r_{\text{settle}}^2 + h^2$, where the right-hand side is precomputed in software and $q_{\min}$ is obtained from the nearest-magnet selection in stage S3b. For the velocity condition, a component-wise test $|v_x| < v_{\text{settle}} \ \& \ |v_y| < v_{\text{settle}}$, is used instead of an Euclidean speed test to avoid the extra DSP cost of computing $v_x^2 + v_y^2$. The settle logic is therefore split across two pipeline stages: S3b selects the minimum q value and corresponding magnet ID, while S3c checks the distance and velocity conditions and updates the consecutive settle counter.

4.5 Band Streaming for High-Resolution Output

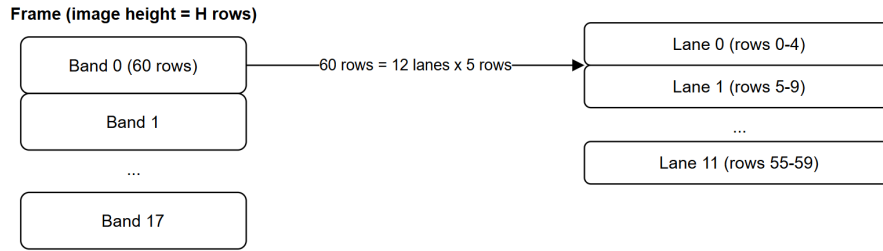


Figure 6: Band streaming approach.

To support high-resolution basin visualisation, the accelerator avoids storing a full output frame in on-chip memory. A full 2048×1080 frame with 6 bits per pixel would require 12.6 Mbit, which exceeds the approximately 4.9 Mbit of BRAM available on the PYNQ-Z1. The full frame therefore cannot be buffered entirely on chip. Instead, the output image is processed in horizontal bands. For the 12-lane design, each band contains

$$\text{BAND_H} = \text{NUM_LANES} \times \text{BAND_ROWS} = 12 \times 5 = 60$$

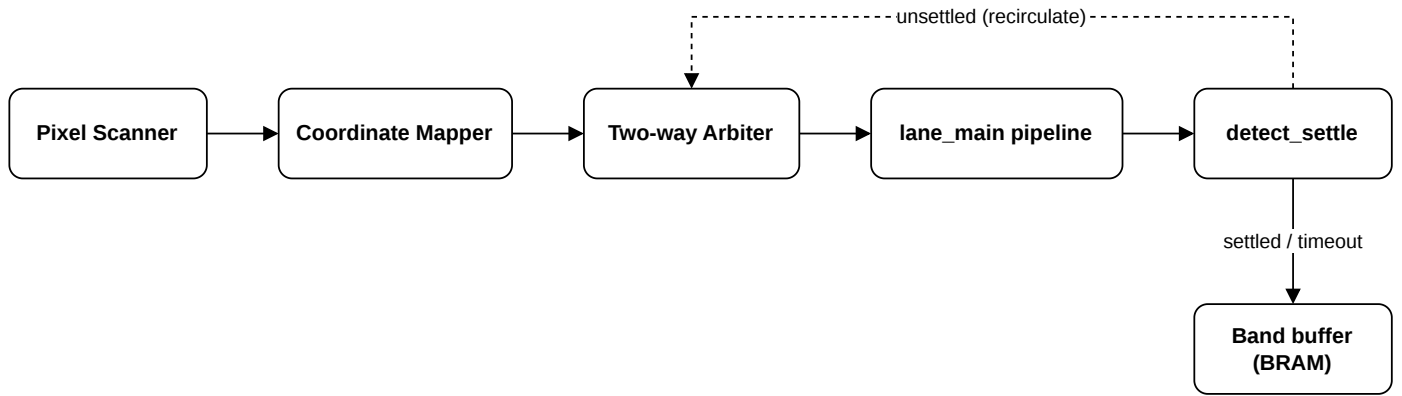
rows, giving $\text{NBANDS} = 1080/60 = 18$ bands for a 2048×1080 frame. Within each band, each lane owns five contiguous rows and writes its results into a small local band buffer.

After all lanes complete a band, the readout FSM streams the band buffer contents over AXI-Stream to DDR memory through AXI-DMA. A one-cycle `band_advance` pulse then releases the lanes to begin the next band. This bounds the required PL buffer memory by the band height rather than the full image height. For example, each lane buffer stores $2048 \times 5 \times 6 = 61.4$ Kbit, which requires approximately two 36 Kbit BRAM blocks. Across 12 lanes, the band buffers therefore use about 24 of the 140 available BRAM blocks, leaving the remaining memory for LUTs and other datapath storage.

This design allows the complete frame to be assembled in DDR, where memory capacity is much larger, while keeping the FPGA-side buffering small. Since each lane owns contiguous rows within a band, reading the lane buffers in order produces pixels in raster order for the final output frame.

4.6 Overall Implementation of One Lane

Since `lane_main` advances a pixel trajectory by only one timestep per pass, the one-lane wrapper must recirculate active pixels until they either settle or reach the timeout limit. A top-level module therefore combines three functions: admitting new pixels from the scanner and coordinate mapper, feeding unsettled pixels back into the lane, and writing completed pixels to the frame buffer.



At the lane input, a priority arbiter selects between a returning unsettled pixel and a newly generated pixel. Feedback pixels are given priority so that active trajectories can continue progressing without requiring a deep FIFO on the feedback path. If a new pixel is generated while the feedback path is occupied, it is held in a single-entry register and the scanner is temporarily stalled. This prevents pixel data from being overwritten.

At the lane output, the termination logic classifies each pixel as settled, timed-out, or still active. A pixel is declared settled when its consecutive settle counter reaches the required threshold, and it is declared timed out when the step count reaches `max_steps` before settling. Active pixels are recirculated through the arbiter, while settled or timed-out pixels are written to the frame buffer.

The frame buffer is implemented as dual-port BRAM and stores a 6-bit result for each pixel,

$$\{\text{step_category}[3:0], \text{magnet_id}[1:0]\}.$$

The lower two bits encode the final or nearest magnet label used for the basin-of-attraction image. The upper four bits quantise the number of timesteps into discrete categories, allowing the software and user interface to generate a settling-time heat map.

4.7 Parallel Solver Architecture

The pixel generator is the top-level hardware module responsible for launching basin-of-attraction computations. It maps each output pixel to an initial pendulum position in the simulation coordinate window, advances the corresponding trajectory until it settles or reaches the timeout, and writes the resulting pixel classifications to the output stream. The module has two main interfaces: an AXI-Lite slave interface for runtime parameters, and an AXI-Stream master interface connected to the AXI-DMA S2MM channel for transferring completed frame data to memory.

The pixel generator is organised into three main parts. The first part is the AXI-Lite control stage, which stores runtime-adjustable parameters in a register file. These include the coordinate window, physical parameters, magnet positions, settling thresholds, maximum step count, and selected output resolution. The second part is the parallel compute stage, which instantiates 12 copies of the one-lane solver described in Section 4.6. Since each pixel trajectory is independent, the band is divided into 12 horizontal slices, and each is assigned to one lane. Lane completion flags are combined so that the full band is marked complete only when the slowest lane has finished.

The third part is the readout stage. Once all lanes have completed their assigned slices as part of the current band, a readout FSM iterates through the frame buffer contents in raster order and streams the results to AXI-DMA. Each pixel produces a 6-bit result containing the magnet label and step-count category. For transfer, each 6-bit value is padded to 8 bits, and four pixels are packed into one 32-bit AXI-Stream word. The readout FSM then pulses `band_advance`, and the above process repeats for all `NBANDS` bands to stream the entire frame.

Table 5: Band-readout FSM.

State	Logic
PS_IDLE	Wait for <code>start</code> ; clear counters.
PS_WAIT	Wait for <code>band_done_all</code> ; initialize read pointers.
PS_ISSUE	Present address; absorb one cycle of BRAM latency.
PS_COLLECT	Latch pixel, pack into word, advance lane pointer; flag <code>tlast</code> on the last pixel of the last band.
PS_SEND	Stall while <code>tready</code> low; emit word, then continue / <code>band_advance</code> / finish.
PS_DONE	Latch <code>frame_done</code> .

A pixel’s position within the overall band is decoded to the owning lane ID and a local buffer address within the lane. The FSM keeps three registers (`rd_lane`, `rd_lane_base`, `rd_next_base`)

updated incrementally. When the read address reaches `rd_next_base` it advances to the next lane and increments the other two registers by `band_pix`, the amount of pixels per lane in one band. This reduces the decode to one comparison and one addition per pixel, avoiding decode through a direct division by `band_pix`, which would require a DSP risk a timing violation.

To respect DMA back-pressure, `PS_SEND` stalls indefinitely while the DMA deasserts `tready`, so no pixel is ever dropped. When a band is fully streamed the FSM pulses a one-cycle `band_advance` and returns to `PS_WAIT`. `tlast` is asserted only on the final word of the final band, so the entire frame arrives as a single DMA transfer.

4.8 AXI-Lite Control Interface

The AXI-Lite slave interface exposes the simulation parameters to the ARM Cortex-A9 processor through memory-mapped registers on the PS GP0 port. AXI-Lite was selected because parameter updates are infrequent and low bandwidth, typically occurring once per frame rather than as a continuous data stream. A lightweight register-based interface is therefore more appropriate than using AXI-Stream or a full AXI4 memory-mapped data path for control.

The register map stores the runtime-adjustable inputs for each frame, including magnet positions, the physical parameter θ , settling thresholds, maximum step count, and control signals for starting or resetting the pipeline. From the software side, the PYNQ Python layer writes these registers using memory-mapped I/O before initiating the DMA transfer. This separates the low-bandwidth control path from the high-bandwidth frame-output path, simplifying both the FPGA logic and the software control flow.

4.9 AXI-DMA Data Transfer

Once a frame has been computed, the frame buffer contents are transferred from the FPGA fabric to DDR memory using AXI-DMA. The `pixel_generator` exposes an AXI-Stream master interface connected to the AXI-DMA S2MM channel, allowing pixel data to be written directly into a pre-allocated DDR buffer without CPU-managed register reads.

For transfer, each result is padded to one byte and unpacked later by the PYNQ Python layer. AXI-DMA was chosen over AXI-Lite because the basin image contains many thousands of pixels; reading each pixel through a register transaction would introduce significant software overhead. DMA therefore provides a more appropriate high-throughput path for frame readout.

VDMA was considered but rejected because it is intended for continuous, multi-buffered video streaming such as direct HDMI output. In this system, frames are generated on demand after each computation trigger and are then passed to the software visualisation pipeline rather than driven directly to a display. A single-transfer AXI-DMA interface is therefore more appropriate and uses fewer FPGA resources. The PYNQ software initiates the transfer, waits for completion, and then reads the result buffer for forwarding to the frontend.

4.10 Trajectory Capture and Plotting

The basin-of-attraction image shows only the final classification of each initial condition, but does not show the transient path followed by the pendulum. To improve the educational value of the visualisation, the system includes a trajectory capture mode for a selected pixel. This allows the frontend to plot the sequence of (x, y) positions taken by the pendulum before it settles or reaches the timeout limit.

Rather than modifying every stage of the lane to explicitly trace one selected pixel, the hardware monitors the output of each lane. Since each lane produces at most one updated pixel state per clock cycle, the trajectory logic compares the output pixel ID with the selected trajectory ID. When

a match occurs, the corresponding fixed-point position and pendulum state are captured.

The captured trajectory samples are stored in a dedicated BRAM as 36-bit words, $\{x[17:0], y[17:0]\}$, with the number of valid samples exposed through an AXI-Lite register. The CPU then reads the stored samples by writing a BRAM read address through AXI-Lite and reading back the corresponding x and y values from two registers. Since a single trajectory contains at most `max_steps` samples, its payload is much smaller than the full basin image. AXI-Lite readout is therefore sufficient, avoiding the extra logic cost of a second DMA path.

5 Interactive Visualisation & Control

5.1 Physical Interaction Layer: ArUco Magnet Tokens

The project aims to provide an educational visualisation of a nonlinear dynamical system. Accordingly, the user interface was designed not only to display the BoA results, but also to make the simulated system easier to understand and interact with. The UI-specific requirements were therefore as follows:

1. Allow users to adjust simulation parameters in an intuitive way;
2. Make the relationship between the physical setup and the simulated system clear;
3. Provide additional visual information beyond the final basin classification.

Physical model parameters, number of magnets and their position are adjustable. To make adjusting parameters more intuitive and interesting, a mixed-reality interaction was adopted where physical tokens represent the magnets. Users move, add or remove these tokens in real life to change the position and number of magnets in the simulation. The simulation directly responds to and mirrors users' physical inputs, while other parameters are adjusted on the Unity frontend. This mixed-reality UI makes the magnetic pendulum simulation feel more tangible and memorable than a computer-only interaction, supporting its goal as an educational tool.

An initial design placed the physical tokens directly on the display, allowing the visualization to form around them. However, this was rejected because the system requires panning and zooming the visualization, which makes the physical token positions inconsistent with the displayed simulation. Therefore, a separate physical board was used for token placement. **[TODO: Purpose of Pan and Zoom to be added in different section]**

5.1.1 Initial Tracking Approach: IMU-Based Tokens

An IMU-based tracking approach was initially considered for locating the physical tokens. In this design, each token would contain an IMU connected to a microcontroller, and the measured horizontal acceleration would be integrated twice to estimate the token position. The estimated positions would then be transmitted to the host computer and mapped to the corresponding magnet coordinates in the simulation.

This approach was experimentally evaluated using an MPU92.65 connected to an ESP32. The first limitation observed was sensitivity to orientation. If the IMU was not perfectly level, a component of gravitational acceleration was projected onto the horizontal measurement axes, producing false displacement even when the token was stationary. To reduce this effect, 200 initial samples were collected at 200 Hz to estimate the static bias, which was then subtracted from subsequent measurements.

However, positional drift remained significant even after bias correction. Since position is obtained by double-integrating acceleration, small measurement noise and residual bias accumulate rapidly over time, leading to large position errors. Although more advanced sensor fusion could reduce drift, this would increase the implementation complexity beyond the project scope. The IMU approach would also require each token to contain an IMU, microcontroller, power source, and wireless communication, making the tokens bulkier and less convenient for users. For these reasons, IMU-based tracking was rejected in favour of a camera-based approach.

Figure placeholder: `./media/IMUsetup.png`
 Add this file beside the `.tex` in the matching media folder to render the image.

Figure 7: IMUsetup.png

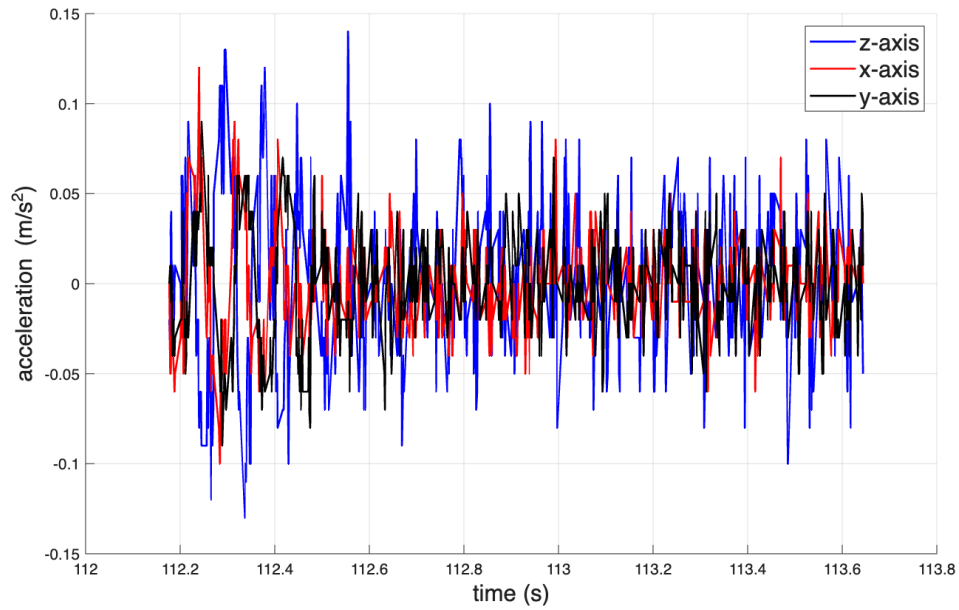


Figure 8: IMUNoise.png

Figure placeholder: `./media/IMUDrfit.png`
 Add this file beside the `.tex` in the matching media folder to render the image.

Figure 9: IMUDrfit.png

5.1.2 Final Tracking Approach: Camera-Based ArUco Detection

Following the limitations of IMU-based tracking, a camera-based computer vision approach was adopted. In the final design, a camera observes a predefined physical area containing the tokens. The detect token positions are then mapped to the simulation coordinate system and used as magnet

positions for FPGA computation. This approach avoids integration drift, keeps the tokens passive, and centralises the tracking logic on the host computer.

Direct object detection was considered, but it produced jitter in the estimated magnet positions even when the tokens were stationary. To improve stability and identification, ArUco markers were attached to the tokens. ArUco markers provide robust square-marker detection and encode unique IDs, allowing each magnet token to be identified independently. The OpenCV ArUco module was used to detect the markers in each video frame, estimate their image-space positions, and provide stable inputs for the simulation.

5.1.3 ArUco Marker CV Pipeline

[TODO: Update diagram to have wired camera feed option, not just Apple's continuity camera]

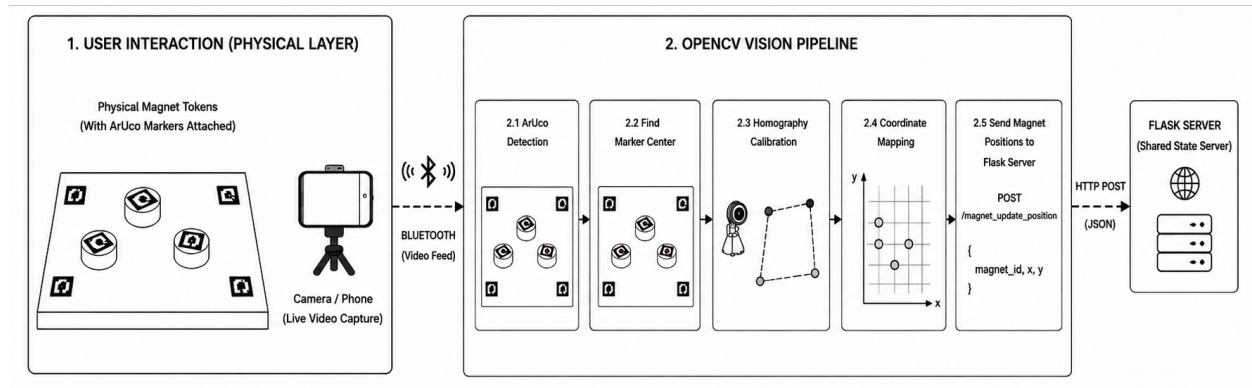


Figure 10: PhysicalLayer.png

Physical magnet tokens are tracked in real time using OpenCV's ArUco marker detection library. Each token carries a printed ArUco marker, while four additional markers are fixed at the board corners to define the simulation coordinate frame. A camera records a continuous video feed of the board, processing each frame individually to detect token positions and convert them into simulation coordinates before sending them to the Flask shared state server.

I. Marker Detection Each frame is converted from BGR to greyscale before marker detection. Since ArUco markers are binary black-and-white patterns, colour information is unnecessary for detection. Processing a single-channel greyscale image reduces the per-frame data volume and improves detection speed. In testing, greyscale detection achieved an average frame-rate improvement of 26.4% compared with colour input, measured using `time.perf_counter()` over three trials.

The `ArucoDetector` object is instantiated once before the detection loop and reused for each frame. The `DICT_4X4_50` dictionary provides 50 unique 4×4 marker patterns. IDs 0–3 are assigned to the four board-corner markers, while IDs 4–6 are assigned to the three magnet tokens. This allows the board frame and the movable magnets to be identified in the same detection pass.

II. Marker Centre Localization For each detected marker, OpenCV returns the four corner coordinates in image space. The marker centre is computed as the intersection of the two diagonals rather than as the arithmetic mean of the four corners. This is more robust under perspective distortion, where a square marker may appear as a general quadrilateral in the camera image.

Let the four detected corners be $\mathbf{p}_0, \mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3$, ordered clockwise from the top-left. The centre $\mathbf{c}$ is obtained from the intersection of the diagonals $\mathbf{p}_0\mathbf{p}_2$ and $\mathbf{p}_1\mathbf{p}_3$:

$$\mathbf{c} = \mathbf{p}_0 + \lambda(\mathbf{p}_2 - \mathbf{p}_0),$$

where

$$\lambda = \frac{(\mathbf{p}_1 - \mathbf{p}_0) \times (\mathbf{p}_3 - \mathbf{p}_1)}{(\mathbf{p}_2 - \mathbf{p}_0) \times (\mathbf{p}_3 - \mathbf{p}_1)}.$$

Here $\times$ denotes the two-dimensional cross product. Degenerate cases, such as nearly parallel diagonals, are rejected using a small numerical tolerance to avoid unstable centre estimates.

III. Homography Calibration The four fixed corner markers define a perspective homography from camera pixel coordinates to the simulation coordinate frame. The destination points are specified directly in simulation units:

$$\mathbf{P}_{\text{dst}} = \begin{pmatrix} -1.8 & 1.8 \\ 1.8 & 1.8 \\ 1.8 & -1.8 \\ -1.8 & -1.8 \end{pmatrix}.$$

This allows `cv2.perspectiveTransform` to map detected token centres directly into the same coordinate system used by the magnetic pendulum model. The corner markers are stored according to their marker IDs rather than detection order, since OpenCV returns detections in image scan order. The homography is computed only when all four board-corner markers are visible. If a corner marker is temporarily occluded, the update is skipped and the system retains the most recent valid coordinate mapping, preventing incomplete detections from corrupting the simulation state.

IV. Coordinate Mapping and Transmission Detected magnet-token centres are transformed using the calibrated homography to obtain their simulation coordinates. These coordinates are then converted to standard Python floating-point values before JSON serialisation and transmission to the Flask server. A second homography is also used to generate a corrected top-down view of the board, providing visual feedback that the physical board calibration is correct.

Before entering the detection loop, the three magnets are registered with the Flask server, with each request retried up to three times before a `RuntimeError` is raised. Within the loop, detected magnet positions are posted for every valid frame. If a token is not detected, a default coordinate is used instead, ensuring that the server always maintains a well-defined state. When the program exits, all three magnets are removed from the server to restore a clean initial state for future runs.

5.2 Python/PYNQ Control Software

The Python control software runs on the processing system (PS) and acts as the bridge between the Unity client and the FPGA accelerator in the programmable logic (PL). It performs three main functions: writing runtime parameters to the accelerator over AXI-Lite, managing DMA transfers from the FPGA frame buffer to DDR memory, and serving the binary TCP protocol used by the Unity frontend.

5.2.1 Hardware Interface

The hardware overlay exposes two main peripherals to the PS: the `pixel_generator` core, controlled through AXI-Lite registers, and an AXI-DMA engine used for frame readout. Both peripherals are accessed from Python using PYNQ’s MMIO interface, which provides direct memory-mapped access to the control and status registers. This low-level interface was useful during development because it allowed individual register values to be inspected and modified directly during debugging.

The `pixel_generator` register file stores the runtime-adjustable parameters required for each computation, as summarised in Table 6. Floating-point values received from the software or Unity interface are converted into the accelerator’s signed Q4.14 format before being written to the PL. This conversion is performed as

$$q(x) = \text{round}(x2^{14}),$$

with saturation applied where required to keep values within the representable fixed-point range.

Table 6: `pixel_generator` AXI-Lite register map.

Register(s)	Field	Purpose
0	control	start / hold the computation
1–2	<code>x_min, y_min</code>	viewport origin
3–4	<code>dx, dy</code>	per-pixel step in x and y
5–10	(x_i, y_i)	magnet positions
11–15	$\gamma, \omega_0^2, h^2, \mu, \Delta t$	physical and numerical parameters
16–18	settling thresholds	distance and velocity limits for convergence
19	$n_{\text{magnets}}, \text{max_steps}$	magnet count and step limit
30	active mask	per-magnet enable bitmask
31–34	<code>W, H, band_pix, nbands</code>	resolution and band-streaming parameters

To increase throughput at larger resolutions, band streaming was implemented. Instead of buffering a full frame on-chip, the PL emits the image in bands of 60 rows, produced by the 12 parallel solver lanes. The `set_resolution()` function writes the image width, height, pixels per band, and band count. These values are computed once on the PS rather than in the PL to reduce LUT usage. As a result, the same bitstream can support resolutions from 160×120 up to 2048×1080 without rebuilding, with the only constraint being that the image height must be a multiple of 60.

5.2.2 DMA Transfer

Frames are returned through the DMA S2MM channel in simple mode, with completion detected by polling. The `arm_dma()` function performs the standard arming sequence: clearing any stale status flags, enabling run/stop and completion interrupt bits, writing the destination address, and finally writing the transfer length, which also starts the DMA transaction. The destination buffer is allocated using `pynq.allocate()` rather than a NumPy array, because the DMA requires physically contiguous memory. The largest required buffer is allocated once during start-up and reused for every frame.

The `poll_dma()` function monitors the DMA status register for the completion bit while also checking for error flags. It returns the elapsed time so that per-frame transfer latency can be recorded. Before the CPU reads the received frame, the buffer cache is invalidated to ensure that the processor reads the data written by the DMA rather than stale cached values.

5.3 Unity Based User Interface

The Unity application provides the user-facing layer of the magnetic pendulum system. Its role is to receive computed basin data from the FPGA, render the results in an interpretable form, and allow users to interact with the simulation parameters without needing to access the low-level hardware interface. The main responsibilities of the interface are:

1. maintain a reliable connection to the FPGA/PYNQ backend;
2. display the basin-of-attraction result using complementary 2D and 3D visualisations;
3. allow users to adjust model parameters with immediate visual feedback;
4. integrate physical magnet tracking from the ArUco-based interaction layer;
5. provide navigation tools for exploring the basin image and trajectory structure.

The Unity codebase was modularised so that each major function is handled by a separate component. Networking, parameter control, rendering, physical magnet display, and camera navigation are therefore developed and tested independently before being integrated into the complete interface. Table 7 summarises the main Unity components and their responsibilities.

Table 7: Unity components and responsibilities.

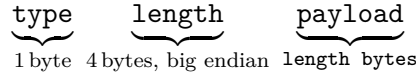
Script	Responsibility
<code>PynqConnection</code>	Background threaded binary TCP client, frame coder/decoder, auto reconnect, outbound parameter/magnet sending, main thread event signaling.
<code>PynqParamController</code>	Pulls and unduplicates slider driven parameter updates before they are sent.
<code>SliderTextDisplay</code>	Maps a normalised UI slider to a physical parameter range, formats the readout, raises change/release events.
<code>PendulumRenderrer</code>	Decodes each pixel into basin and iteration count), builds the 2D category and value textures and the 3D height field mesh.
<code>MagnetRenderrer</code>	Polls the AWS hosted Flask vision endpoint for magnet positions, draws them on a mini map, and relays them to the FPGA (using the custom TCP protocol).
<code>PanZoom</code>	Pan/zoom of the 2D simulation window, constrained to the valid simulation bounds. [TODO: Krish add network]
<code>OrbitCamera</code>	Orbit/pan/zoom camera for the 3D surface view.
<code>ViewManager</code>	Switches between the 2D and 3D representations of the basin image.
<code>ImagePostToFrameTimer</code> , <code>SliderToImageTimer</code>	Latency test scripts (local render time and end to end round trip).
<code>VertexColor</code> (shader)	Minimal URP shader that renders the 3D mesh using per-vertex basin colours.

5.3.1 Networking Layer and Wire Protocol

Latency-critical communication between Unity and the PYNQ backend uses a single persistent TCP connection managed by `PynqConnection`. A custom binary framing protocol was chosen over

JSON or HTTP for image transfer because each basin frame consists of dense numerical data that may need to be transmitted repeatedly. Sending the frame as raw bytes avoids repeated connection setup and removes the overhead of converting large numerical arrays to and from text formats.

Each message follows the same structure:



The outbound message types are **PARAMS** (0x01) and **MAGNETS** (0x02), while inbound messages are **IMAGE** (0x10) and **INFO** (0x11). An image payload begins with a 9-byte header containing the frame width, height, bit depth, and version number, followed by the packed pixel data. Parameter and magnet messages use compact JSON payloads inside the same binary framing structure, as these messages are small and less latency-critical than full image frames.

I. Threading Networking is handled on a dedicated background thread so that blocking socket reads do not stall Unity’s render loop. Decoded messages are pushed into a lock-free **ConcurrentQueue**, which is drained in **Update()** on the Unity main thread. This producer–consumer structure ensures that received frames can be parsed asynchronously, while Unity objects such as **Texture2D** and **Mesh** are updated only from the main thread, where it is safe to do so.

The connection also includes a recovery mechanism. If a socket error occurs, the stream is closed, the client waits for a configurable delay, and the connection is re-established. Any pending parameter frame is resent after reconnection, preventing temporary network failures from leaving the frontend and accelerator in an inconsistent state.

II. Frame versioning Each parameter update carries an incrementing version number. After sending an update, the Unity client records the minimum image version it is willing to display. Incoming frames older than this watermark are discarded by **PendulumRenderer**. This prevents the interface from briefly showing a stale basin image that was generated before the latest slider change took effect, which is especially important when the user is dragging a parameter slider and several frames may still be in transit.

III. Wire Protocol Summary

Table 8: Summary of the Unity–PYNQ binary wire protocol. Each message contains a 1-byte type field, a 4-byte big-endian length field, and a variable-length payload.

Type	Direction	Payload
0x01 PARAMS	UI→FPGA	JSON: version, dampingFactor, magneticStrength, pendulumLength, pendulumHeight
0x02 MAGNETS	UI→FPGA	JSON: { magnets: { magnet_i: {x,y} } }
0x10 IMAGE	FPGA→UI	9-byte header (w,h,depth,version) + packed pixels
0x11 INFO	FPGA→UI	JSON: magnet dictionary

5.3.2 Parameter Control Interface

Damping factor, magnet strength, pendulum length, and pendulum height are exposed to the user as sliders. The **SliderTextDisplay** component maps each slider’s normalised $[0, 1]$ value to its corresponding physical parameter range $[\text{min}, \text{max}]$. It also rounds the displayed value to two decimal places for readability and raises two events: one when the slider value changes, and another when the user releases the slider.

A naive implementation would transmit a parameter update on every slider callback, which could flood the network link and repeatedly trigger the accelerator while the user is dragging a control. To avoid this, `PynqParamController` rate-limits parameter updates to one every 0.1 s during dragging, while forcing an immediate final update when the slider is released. It also removes duplicate updates: if the current parameter snapshot matches either the last sent value or a value already queued for transmission, it is discarded. This provides responsive interaction without unnecessarily saturating the FPGA control path, while ensuring that the final steady-state parameter values exactly match the UI. The throttle, deduplication, and forced-send behaviour form a deliberate latency-throughput trade-off, discussed further in Section 5.3.6.

5.3.3 Basin Visualisation

The `PendulumRenderer` component subscribes to `ImageReceived` events and converts each received frame into multiple visual representations. Each pixel encodes two quantities: the basin label, indicating which magnet the pendulum settles near, and an iteration-count category, indicating how long the trajectory takes to settle. For the current 6-bit hardware output, the lower two bits store the basin label and the upper four bits store the iteration category:

$$\text{category} = \text{pixel} \& 0xF, \quad \text{depth} = (\text{pixel} \gg 2) \& 0xF.$$

This bit-depth-aware decoding allows the same rendering pipeline to support the current output format while remaining adaptable to future hardware formats with higher bit depth.

The renderer converts each decoded frame into three complementary visualisations. First, the basin label is mapped to a small colour palette to produce the standard 2D basin-of-attraction image, with one colour per magnet and a separate colour for unresolved pixels. Second, the iteration-count category is normalised and displayed using a colour gradient, highlighting regions that take longer to settle and making sensitive basin boundaries more visible. Finally, the same data is used to construct a 3D height-field mesh, where pixel position defines the horizontal coordinates, iteration count defines height, and basin label defines vertex colour. This provides both a direct classification view and a more informative representation of convergence behaviour.

5.3.4 Physical Token Tracking Integration

`MagnetRenderer` links the physical token board to the simulated magnetic pendulum system. It polls the vision endpoint every 0.1 s (default) for the latest magnet positions, maps each simulation coordinate in the range $[-1.8, 1.8]$ onto a 130×130 mini-map, and displays the magnets as coloured markers. The same coordinates are then forwarded to the FPGA over TCP using `SendMagnets`.

The vision pipeline obtains these coordinates from the ArUco tracking system. Four fixed corner markers, with IDs 0–3, define the homography used to rectify the camera view to the square simulation board, while marker IDs 4–6 correspond to the movable magnet tokens. Their detected centres are transformed into simulation coordinates before being published. Since Unity both displays the detected magnet positions and forwards them to the FPGA, the on-screen magnet markers remain consistent with the basin image computed by the accelerator.

5.3.5 Navigation & View Management

The Unity interface provides separate navigation controls for the 2D basin view and the 3D height-field view. In the 2D mode, `PanZoom` allows the user to pan across the basin image and zoom into regions of interest, while constraining the view to the valid simulation bounds. This is useful for inspecting fine basin-boundary structure and local regions where the final magnet classification changes rapidly.

In the 3D mode, `OrbitCamera` provides orbit, pan, and zoom controls around the height-field visualisation, allowing the user to inspect the relationship between basin labels and convergence

time from different viewpoints. The `ViewManager` switches between the 2D canvas and the 3D overlay, enabling only the relevant camera controls for the active view. This separation prevents input conflicts and keeps the interaction model consistent for each visualisation mode.

5.3.6 Design Decisions & Trade-offs

Several design decisions were made to balance responsiveness, robustness, and implementation complexity in the Unity interface.

- **Binary TCP for frame transfer.** Basin frames contain dense pixel data, so they are transmitted as packed bytes over a persistent TCP connection. This avoids repeated per-frame connection setup and the overhead of serialising large numerical arrays into text formats. Lower-rate messages, such as parameter and magnet updates, remain as compact JSON payloads inside the same framing structure for convenience.
- **Persistent wired TCP connection.** An initial Flask-based approach was tested for transferring image and parameter data, but latency measurements using `ImagePostToFrameTimer` and `SliderToImageTimer` showed highly variable end-to-end delays, ranging from approximately 500 ms to 1.5 s. This was caused by the server handling multiple requests while also transferring large image payloads. A FastAPI WebSocket approach and a wired TCP connection were then considered, with the wired TCP approach providing the lowest and most stable latency. It was therefore adopted for the final interface.
- **Background I/O and main-thread marshalling.** Network communication runs on a background thread so that blocking socket operations do not stall Unity's render loop. Decoded messages are then passed to the Unity main thread before updating objects such as textures and meshes, respecting Unity's threading model.
- **Throttled and deduplicated parameter updates.** Slider updates are rate-limited while the user is dragging to avoid flooding the communication link or repeatedly triggering unnecessary accelerator recomputations. A final update is forced when the slider is released, ensuring that the hardware state matches the final UI value.
- **Frame versioning watermark.** Each parameter update carries a version number, and frames older than the latest accepted version are discarded. This prevents outdated basin images from briefly appearing after a user changes a parameter.
- **Bit-depth-aware decoding.** The renderer supports the current 6-bit hardware output while remaining adaptable to future n -bit formats. This decouples the Unity rendering pipeline from the exact FPGA output bit width.
- **Multiple visual encodings.** The category map shows which magnet each initial condition converges to, while the value map and 3D height-field visualisation show convergence time and boundary sensitivity. This supports the educational aim of the interface by showing both final classification and dynamic behaviour.

6 Verification and Evaluation

This section evaluates the system from four perspectives: numerical accuracy, hardware feasibility, runtime performance, and end-to-end responsiveness. Unless otherwise stated, all tests use the nominal parameter set defined in Section 3.5. Numerical accuracy is evaluated against the floating-point reference, while hardware performance is assessed using timing closure, resource utilisation, runtime, latency, and energy consumption.

6.1 Numerical Accuracy Evaluation

6.1.1 Match Rate Analysis

The main accuracy metric is pixel-level match rate. For each initial condition, the output label from the fixed-point model or FPGA is compared with the corresponding label from the floating-point reference. The match rate is defined as

$$\text{Match rate} = \frac{N_{\text{match}}}{N_{\text{total}}} \times 100\%,$$

where N_{match} is the number of pixels whose final magnet label matches the floating-point reference, and N_{total} is the total number of pixels being compared.

I. Fixed-Point Format Implementing floating-point arithmetic on the FPGA would require substantially more LUTs and flip-flops, limiting the number of parallel solver lanes. Since the pendulum state remains within a bounded numerical range, fixed-point arithmetic is sufficient and maps efficiently onto the DSP48E1 25×18 -bit multiplier. It also provides deterministic arithmetic, which is important for bit-exact verification against the fixed-point software model.

The fixed-point sweep experiment was performed to quantify how word length, fractional precision, and LUT size affect agreement with the floating-point reference, allowing the final hardware format to be selected as an accuracy–resource trade-off.

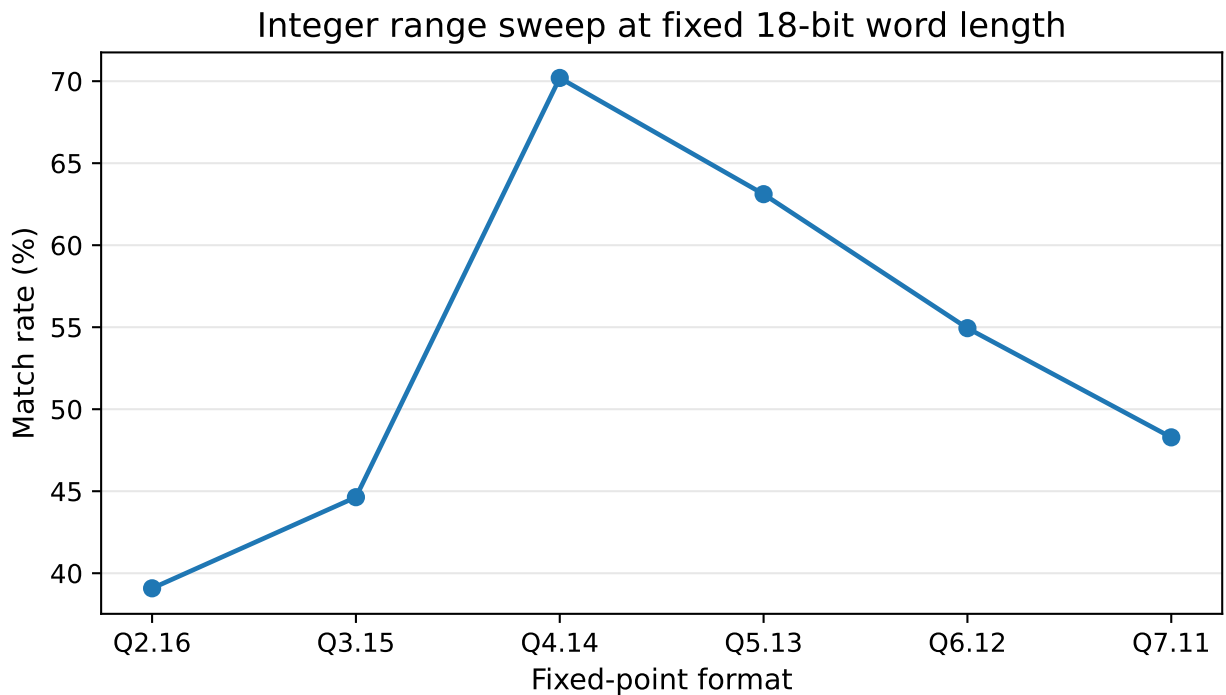


Figure 11: Integer bit sweep experiment.

LUT & Multiplier Design Stage S4 of the solver approximates the nonlinear inverse-distance term $q^{-3/2}$, where

$$q_i = \Delta x_i^2 + \Delta y_i^2 + h^2.$$

In the floating-point reference model this expression is evaluated directly, but a direct FPGA implementation would require square-root and division operations. Since the term must be evaluated once per magnet per timestep, implementing it directly would significantly increase latency and resource usage in each solver lane.

To avoid this cost, the final design replaces the direct computation of $q^{-3/2}$ with a precomputed lookup table implemented in BRAM. The table contains 4096 fixed-point samples, each stored as an 18-bit value. This converts the nonlinear operation into a synchronous memory read, trading a small amount of BRAM for a much simpler and faster datapath.

The LUT is indexed using the offset of q from its minimum possible value. Since $q \geq h^2$, the address is formed from

$$q_{\text{diff}} = q - h^2.$$

The value q is represented in unsigned Q5.13 format, while h^2 , originally in Q4.14, is converted to Q5.13 by a one-bit right shift. The LUT index is then computed as

$$\text{idx} = \text{clip} \left(\left\lfloor \frac{q_{\text{diff}}}{2^{-9}} \right\rfloor, 0, 4095 \right).$$

Equivalently, in the RTL this is implemented by right-shifting the Q5.13 difference by four bits, since $2^4 \times 2^{-13} = 2^{-9}$. Therefore, each LUT entry corresponds to a q -step of $\Delta q = 8/4096 = 2^{-9}$, and the table spans the interval $[h^2, h^2 + 8)$.

This windowed address range concentrates LUT resolution near small values of q , where $q^{-3/2}$ changes most rapidly and the magnetic force is most sensitive. Values outside the represented range are clamped to the final table entry. No interpolation is used: a floor lookup is chosen so that the LUT stage remains a single-cycle synchronous memory read. Although interpolation could reduce approximation error, it would require additional arithmetic in the critical path. The accuracy and resource trade-off of this LUT design is evaluated as follows:

Table 9: Performance comparison across variants.

Rank	Variant	Strict	All	Both-Strict	Fixed Strict
1	exact_inverse_force_plus_rounded	81.67%	77.84%	83.67%	90.81%
2	piecewise_small_q_lut_plus_rounded	81.53%	77.83%	83.52%	90.62%
3	shifted_lut_range8_plus_rounded	81.24%	77.46%	83.22%	91.01%
4	shifted_lut_range16_plus_rounded	80.24%	76.57%	82.16%	91.13%
5	piecewise_small_q_lut	78.56%	74.35%	80.71%	90.79%
6	shifted_lut_range8	77.94%	73.79%	79.90%	91.15%
7	rounded_multiplier	77.33%	73.68%	79.18%	91.45%
8	baseline_current_hw	73.94%	70.20%	75.89%	91.48%

6.2 Hardware Feasibility Evaluation

6.2.1 Timing Closure

The implemented design meets all timing constraints at the 100 MHz target clock, with a worst negative slack of +0.034 ns ($f_{\max} \approx 100.3$ MHz), a worst hold slack of +0.013 ns, and no failing endpoints. The small margins validate the datapath pipelining choices.

6.2.2 Resource Utilisation

Figure 12 reports the post-implementation utilisation on the PYNQ-Z1.

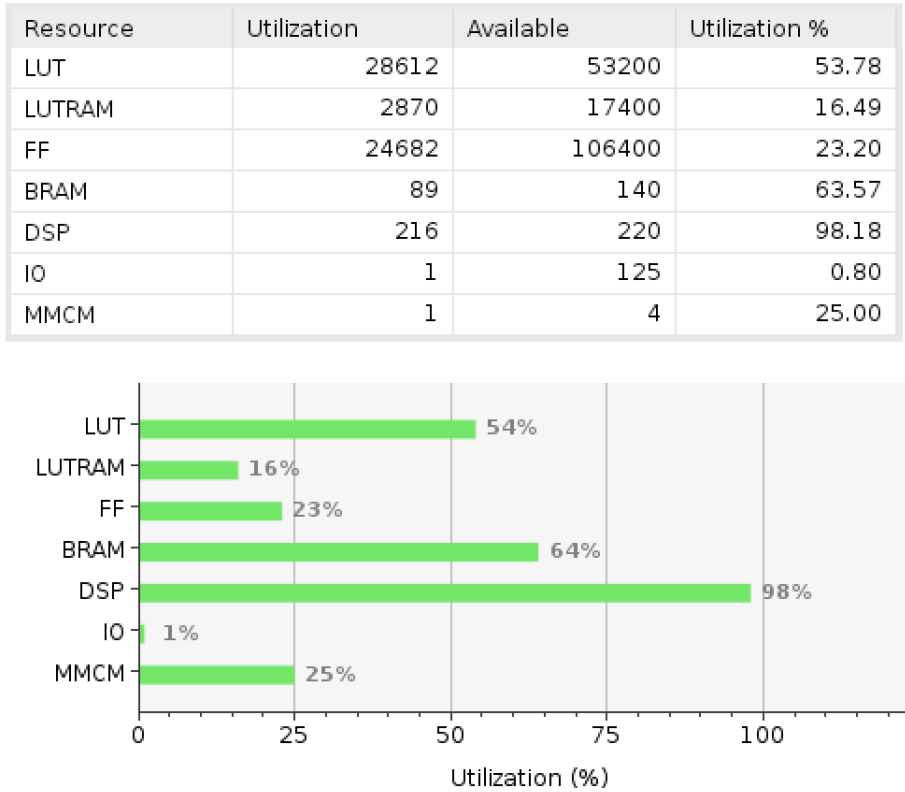


Figure 12: Vivado Utilisation Report.

The design is DSP-bound: with 18 DSP per lane, 12 lanes is the maximum the board can support. This validates the DSP-reduction decisions in `fx_divide_by_100` and `coord_mapper`, allowing us to increase the number of lanes and hence throughput. By contrast, BRAM sits at only 63.6%, confirming that band streaming keeps on-chip memory well within budget. Given the logic resource utilisations (LUTs 53.8%, FFs 23.2%), the accelerator is limited by arithmetic rather than logic. The spare BRAM cannot be used for additional lanes due to DSP limitations. Another possible use of the free BRAM is enlarging the $q^{-3/2}$ ROMs for finer resolution. This is blocked by BRAM limitations, as doubling each ROM to 8192×18 would increase BRAM usage to 4 tiles, so 36 ROMs would require 144 tiles, more than 140 available.

6.3 Runtime Performance Evaluation

6.3.1 Cross-Platform Performance Comparison

```

xilinx@pyng:~/jupyter_notebooks$ ./cpu_fixedpoint
Fixed-point magnetic pendulum (C++ baseline)
resolution      : 270 x 270
fixed format    : signed Q4.14
raw range       : [-131072, 131071]
force mode      : lut-force
gamma, omega2, mu : 0.2, 1, 1
dt, max steps   : 0.01, 4096
settling thresholds : r=0.25, v=0.05

step 1/4096 | active pixels 72900 | elapsed 1.01246s
step 500/4096 | active pixels 65402 | elapsed 470.457s
step 1000/4096 | active pixels 59163 | elapsed 908.072s
step 1500/4096 | active pixels 46204 | elapsed 1278.81s
step 2000/4096 | active pixels 30796 | elapsed 1557.44s
step 2500/4096 | active pixels 14983 | elapsed 1727.56s
step 3000/4096 | active pixels 6510 | elapsed 1812.37s
step 3500/4096 | active pixels 5850 | elapsed 1871.3s
step 4000/4096 | active pixels 5837 | elapsed 1929.04s
step 4096/4096 | active pixels 0 | elapsed 1940.13s

Done in 1940.13s
Pixels per second: 37.5748
Average time per pixel: 26.6136 ms

=== Results ===
Format      : Q4.14
Resolution  : 270 x 270
Elapsed     : 1940.13s
Pixels/second : 37.5748
ms/pixel    : 26.6136
Strict settled : 67064 / 72900 (91.9945%)
Unresolved (-1) : 0
Magnet 0 pixels : 21942
Magnet 1 pixels : 23183
Magnet 2 pixels : 27775

```

Figure 13: CPU baseline simulation results.

To evaluate the effectiveness of the accelerator, we compared the FPGA implementation against a software baseline running on the PYNQ processor. The CPU implementation required approximately 1940.13 seconds, or around 32.3 minutes, to generate a 270×270 boa image. In contrast, the FPGA completed the same task in [insert] at 110 MHz, resulting in an overall speed-up of approximately [insert]. The FPGA also demonstrated significantly improved energy efficiency. Based on the measured power consumption, the CPU execution consumed approximately 1,500 joules, whereas the FPGA required only 0.199 joules, representing an energy reduction of over $7,500\times$. These results demonstrate the benefits of exploiting the inherent parallelism of the magnetic pendulum simulation through a custom FPGA architecture.

As seen in Figure 14, the board uses 2.782 W.

Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 2.782 W
Design Power Budget: Not Specified
Process: typical
Power Budget Margin: N/A
Junction Temperature: 57.1°C
 Thermal Margin: 27.9°C (2.3 W)
 Ambient Temperature: 25.0 °C
 Effective θ_{JA} : 11.5°C/W
 Power supplied to off-chip devices: 0 W
 Confidence level: Low

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

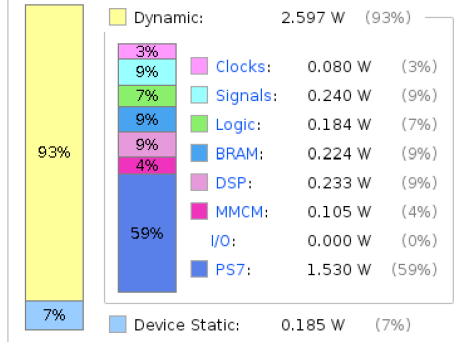
On-Chip Power

Figure 14: Vivado Power analysis.

Platform	Time (s)	Throughput (Mpx/s)	Speed-up (×CPU)	Power (W)	Energy /frame (J)
160 × 120 (19,200 px)					
CPU	[...]	[...]	1×	[...]	[...]
GPU	0.0025	7.59	[...]	39.8	0.101
FPGA	0.071	0.27	[...]	2.78	0.197
1024 × 540 (552,960 px)					
CPU	[...]	[...]	1×	[...]	[...]
GPU	0.0272	20.34	[...]	39.8	1.082
FPGA	0.244	0.25	[...]	2.78	0.68
2048 × 1080 (2,211,840 px)					
CPU	[...]	[...]	1×	[...]	[...]
GPU	0.0958	23.08	[...]	39.8	3.814
FPGA	7.10	0.31	[...]	2.78	19.74

Table 10: Cross-platform comparison across resolutions (CPU: [fill in info]; GPU: RTX 4060 Laptop, fp32; FPGA: PYNQ-Z1, @ 100 MHz).

Platform	Throughput (Mpx/s)	Power (W)	Price (£)	Throughput/Power (Mpx/s/W)	Throughput/Price (kpx/s/£)
160×120	<i>(19,200 px)</i>				
CPU	[...]	[...]	[...]	[...]	[...]
GPU	7.59	39.8	290	0.19	26.2
FPGA	0.27	2.78	190	0.10	1.42
1024×540	<i>(552,960 px)</i>				
CPU	[...]	[...]	[...]	[...]	[...]
GPU	20.34	39.8	290	0.51	70.1
FPGA	0.25	2.78	190	0.09	1.32
2048×1080	<i>(2,211,840 px)</i>				
CPU	[...]	[...]	[...]	[...]	[...]
GPU	23.08	39.8	290	0.58	79.6
FPGA	0.31	2.78	190	0.11	1.63

Table 11: Platform efficiency across resolutions.

6.4 Resolution and Latency Trade-Off

6.4.1 Resolution Scaling

Resolution	Pixels	Frame time (s)	Throughput (Mpx/s)
160×120	19,200	0.071	0.27
320×240	76,800	0.244	0.31
640×360	230,400	0.761	0.30
960×540	518,400	1.663	0.31
1280×720	921,600	2.930	0.31
1600×900	1,440,000	4.563	0.32
1920×1080	2,073,600	6.520	0.32
2048×1080	2,211,840	7.066	0.31

Table 12: FPGA resolution scaling.

6.4.2 End-to-End Latency